TaskFlow — Görev ve Proje Yönetim Sistemi REST API

Backend Geliştirme Proje Envanteri — Proje Tanıtım ve Tasarım Çıktısı

Eğitim:	Node.js ile Backend Programlama — Bitirme Projesi
Geliştirici:	Sefa Kesen
GitHub Deposu:	https://github.com/sefakesen/taskflow
Teknolojiler:	Node.js, Express.js, RESTful API, Katmanlı Mimari, JSON Veri Deposu
Tarih:	Ekim 2026

---

1. Projenin Amacı ve Özeti

TaskFlow, yazılım geliştirme ekiplerinin görevlerini, projelerini ve çalışan sorumluluklarını uçtan uca takip edebilmesi için tasarlanmış kurumsal standartlarda bir RESTful API arka yüzüdür. Proje; görev oluşturma, çalışanlara atama, durum takibi, önceliklendirme, arama/filtreleme, veri doğrulama ve temel raporlama servislerini modüler ve katmanlı bir mimaride sunar.

2. Sistem Senaryosu ve İş Akışı

Bir yazılım şirketindeki temel roller ve API kullanım senaryoları şu şekildedir:

- **Proje Yöneticisi:** Sisteme yeni görevler ekler (`POST /api/tasks`), önceliklerini (`low` , `medium` , `high` , `critical`) belirler ve çalışanlara (`musa` , `ayse` , `ali` , `zeynep`) atar.
- **Geliştirici:** Kendisine atanan görevleri listeler (`GET /api/tasks/assignee/:name`), görev detayını görüntüler (`GET /api/tasks/:id`) ve ilerleme durumunu günceller (`PUT /api/tasks/:id`).
- **Ekip Lideri / Yönetim:** Görevleri filtreler, anahtar kelimeyle arar (`GET /api/tasks/search?keyword=...`), raporlama servisleri üzerinden sistem özetini (`GET /api/reports/summary`) ve bekleyen işleri (`GET /api/reports/pending`) izler.

3. Sistem Mimarisi

Uygulama, sorumlulukların ayrılması (Separation of Concerns) prensibi doğrultusunda katmanlı mimariyle yapılandırılmıştır:

```
[İstemci / Postman]
       │
       ▼
[Express JSON Body Parser] (express.json, express.urlencoded)
       │
       ▼
[Logger Middleware] (Konsol ve logs/requests.log dosyasına kayıt)
       │
       ▼
[Router Katmanı] (taskRoutes.js ve reportRoutes.js - modüler rotalar)
       │
       ▼
[Validation Middleware] (Girdi tipi ve sınır doğrulaması)
       │
       ▼
[Controller Katmanı] (taskController.js ve reportController.js - iş mantığı)
       │
       ▼
[Data Store Katmanı] (taskStore.js: Bellek içi hızlı erişim + tasks.json dosya eşitlemesi)
```

4. Veri Modeli Tasarımı (Task Modeli)

Alan	Tip	Zorunlu	Açıklama ve Kısıtlar
`id`	String (UUID v4)	Otomatik	Benzersiz görev tanımlayıcısı (UUID v4)
`title`	String	Evet	Görev başlığı (3 - 100 karakter arası)
`description`	String	Hayır	Görevin detaylı açıklaması
`status`	String (Enum)	Hayır	Durum: `todo`, `in-progress`, `completed`, `cancelled` (Varsayılan: `todo`)
`priority`	String (Enum)	Evet	Öncelik: `low`, `medium`, `high`, `critical`
`assignee`	String	Hayır	Atanan çalışanın kullanıcı adı (En az 2 karakter)
`createdAt`	ISO 8601 String	Otomatik	Görevin oluşturulma zaman damgası
`updatedAt`	ISO 8601 String	Otomatik	Son güncelleme zaman damgası

5. API Endpoint Listesi

Metot	Endpoint	Kapsam	Açıklama
GET	`/api/tasks`	Zorunlu CRUD	Tüm görevleri listeler
POST	`/api/tasks`	Zorunlu CRUD	Yeni görev oluşturur (201 Created)
GET	`/api/tasks/:id`	Zorunlu CRUD	ID ile belirli görevin detayını getirir
PUT	`/api/tasks/:id`	Zorunlu CRUD	Görevi günceller (200 OK)
DELETE	`/api/tasks/:id`	Zorunlu CRUD	Görevi sistemden siler (200 OK)
GET	`/api/tasks?status=...`	İsteğe Bağlı 01	Duruma göre filtreleme
GET	`/api/tasks?priority=...`	İsteğe Bağlı 01	Önceliğe göre filtreleme
GET	`/api/tasks/search?keyword=...`	İsteğe Bağlı 01	Başlık ve açıklamada arama
GET	`/api/tasks/assignee/:name`	İsteğe Bağlı 01	Çalışana göre görev listeleme
GET	`/api/tasks?page=1&limit=10`	İsteğe Bağlı 01	Sayfalama desteği (Pagination)
GET	`/api/tasks?sort=createdAt&order=desc`	İsteğe Bağlı 01	Sıralama desteği (Sorting)
GET	`/api/reports/summary`	İsteğe Bağlı 03	Genel sistem özeti ve istatistikleri
GET	`/api/reports/pending`	İsteğe Bağlı 03	Bekleyen görevlerin sayısı ve listesi
GET	`/api/reports/completed`	İsteğe Bağlı 03	Tamamlanan görevlerin sayısı ve listesi

6. Middleware ve Güvenlik Mimarisi

- **Logger Middleware (Zorunlu — Aşama 03):** Her gelen HTTP isteğinin metodunu, adresini, durum kodunu ve zaman damgasını yakalar. Konsola ve `logs/requests.log` dosyasına kalıcı olarak yazar.
- **Validation Middleware (İsteğe Bağlı Pratik 02):** Gelen veriyi denetler; eksik zorunlu alanlarda veya geçersiz enum değerlerinde açıklayıcı hata mesajlarıyla birlikte `400 Bad Request` döner.
- **Global Error Handler Middleware:** Yakalanmayan hataları ele alarak sistem kararlılığını korur ve standart JSON hata formatı üretir.

7. Test ve Doğrulama Kanıtları

Sistem üzerindeki 5 temel CRUD ucu Postman ile doğrulanmış ve ekran görüntüleri `docs/screenshots/` dizinine eklenmiştir:

- `01_gorev_listeleme.png` — GET /api/tasks (200 OK)
- `02_gorev_ekleme.png` — POST /api/tasks (201 Created)
- `03_gorev_detayi.png` — GET /api/tasks/:id (200 OK)
- `04_gorev_guncelleme.png` — PUT /api/tasks/:id (200 OK)
- `05_gorev_silme.png` — DELETE /api/tasks/:id (200 OK)

Tüm istekleri içeren Postman v2.1 test koleksiyonu `docs/TaskFlow.postman_collection.json` dosyasında sunulmuştur.